Machine Learning Theory & Practical

Master of Computer Application
Semester – II

By
Name: ANUVID MISHRA

Reg no.: 2511022250036

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106

May 2026

INDEX

Sno.	Particulars	Page No.
1	Introduction	1
2	Problem Statement	2
3	Dataset	3
4	System Architecture	4
5	Implementation Details	5
6	Model Training	10
7	Evaluation	11
8	Model Details	12
9	Limitations & Future Work	13
10	Reproduction Guide	14

CHAPTER 1

INTRODUCTION

This document describes the end-to-end development of a sarcasm detection system built on BERT (Bidirectional Encoder Representations from Transformers). The model is fine-tuned on a labelled dataset of Reddit comments to classify text as either sarcastic or non-sarcastic.

Key Takeaway

BERT's deep contextual understanding of language makes it well-suited for sarcasm detection, a task where surface-level word choice is often misleading. Fine-tuning a pre-trained BERT model allows the system to leverage vast prior knowledge of language semantics while specialising on the nuances of ironic expression.

The pipeline covers data ingestion, pre-processing, tokenisation, dataset construction, model configuration, training, and evaluation — all implemented in Python using the HuggingFace Transformers library and PyTorch.

CHAPTER 2

PROBLEM STATEMENT

2.1 Background

Sarcasm is a form of verbal irony where the intended meaning is often the opposite of the literal words spoken or written. Detecting sarcasm in text is a well-known challenge in Natural Language Processing (NLP) because:

- Sarcastic statements often contain positive words with negative intent (e.g., 'Oh great, another Monday').
- Context, tone, and world knowledge are typically required to identify sarcasm — none of which are captured by simple keyword matching.
- Traditional ML models based on bag-of-words or TF-IDF features fail to capture the contextual signals needed.

2.2 Objective

Build a binary classifier that can detect whether a given text comment is sarcastic (label = 1) or non-sarcastic (label = 0), leveraging a pre-trained large language model.

2.3 Use Cases

- Social media sentiment analysis platforms
- Customer feedback and review mining systems
- Content moderation and toxicity detection pipelines
- Conversational AI and chatbot systems requiring intent understanding

CHAPTER 3

DATASET

3.1 Source

The dataset used is a modified version of the Sarcasm on Reddit dataset, stored as modified_sarcasm_dataset.csv. It consists of comments scraped from Reddit, annotated with binary sarcasm labels.

3.2 Structure

Column	Type	Description
comment	String	The raw text of a Reddit comment
label	Integer (0/1)	0 = Non-Sarcastic, 1 = Sarcastic

3.3 Pre-processing

- The 'comment' column is cast to string to handle any null or non-string values.
- Labels are extracted as a Python list of integers for compatibility with PyTorch.
- An 80/20 stratified train-test split is applied (random_state=42 for reproducibility).

3.4 Data Splitting

Split	Proportion	Purpose
Training Set	80%	Used to fine-tune BERT model weights
Test Set	20%	Held out for final evaluation only

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Pipeline Overview

The project follows a standard supervised learning pipeline with transformer-based components:

Stage	Component	Technology
Data Loading	CSV ingestion + column extraction	pandas
Data Splitting	80/20 train-test split	scikit-learn
Tokenisation	Sub-word tokenisation with special tokens	BertTokenizer
Dataset Construction	Custom PyTorch Dataset class	torch.utils.data.Dataset
Model Definition	BERT + classification head	BertForSequenceClassification
Training	Supervised fine-tuning with AdamW	HuggingFace Trainer API
Inference	Predict logits on test set	Trainer.predict()
Evaluation	Accuracy, Precision, Recall, F1	scikit-learn metrics

CHAPTER 5

IMPLEMENTATION DETAILS

5.1 Dependencies

```
import pandas as pd

import torch

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report

from transformers import BertTokenizer, BertForSequenceClassification

from transformers import Trainer, TrainingArguments
```

Library	Version	Role
torch (PyTorch)	>=1.9	GPU/CPU tensor operations, model backend
transformers (HuggingFace)	>=4.x	BERT model, tokenizer, Trainer API
pandas	>=1.3	Data loading and manipulation
scikit-learn	>=0.24	Train-test split and evaluation metrics

5.2 Tokenisation

The BertTokenizer (bert-base-uncased) converts raw text into input IDs and attention masks using WordPiece sub-word tokenisation. Key parameters:

Parameter	Value	Rationale
padding	True	Pads all sequences to the same length within batch
truncation	True	Truncates sequences exceeding max_length
max_length	128	Covers most Reddit comments within BERT's 512-token limit
return_tensors	'pt'	Returns PyTorch tensors directly

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')


def tokenize(texts):
    return tokenizer(
        texts,
        padding=True,
        truncation=True,
        max_length=128,
        return_tensors='pt'
    )
```

5.3 Custom Dataset Class

A custom PyTorch Dataset wraps the tokenised encodings and labels to interface with the HuggingFace Trainer:

```
class SarcasmDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)
```

5.4 Model Configuration

BERT for Sequence Classification is loaded with a two-neuron classification head on top of the [CLS] token representation:

```
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2       # Binary: 0=Non-Sarcastic, 1=Sarcastic
)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
```

GPU Acceleration

The model is automatically moved to a CUDA GPU if available. Training on GPU reduces epoch time from hours to minutes. On CPU, training may be significantly slower depending on dataset size.

5.5 Training Configuration

Hyperparameter	Value	Description
num_train_epochs	2	Number of full passes over the training data
per_device_train_batch_size	8	Samples processed per GPU/CPU per step
per_device_eval_batch_size	8	Evaluation batch size
logging_steps	50	Log training loss every 50 optimisation steps
output_dir	./results	Directory to save checkpoints and model weights
report_to	none	Disables external logging (W&B, TensorBoard)

```
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=2,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    logging_dir='./logs',
    logging_steps=50,
    disable_tqdm=False,
    report_to='none'
)
```

CHAPTER 6

MODEL TRAINING

6.1 Training Process

The HuggingFace Trainer abstracts the training loop, handling gradient computation, backpropagation, and optimisation automatically:

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset
)


trainer.train()
```

Internally, the Trainer uses AdamW (Adam with decoupled weight decay) as the default optimiser with a linear learning rate schedule and warmup steps proportional to training size.

6.2 What Happens During Fine-Tuning

- BERT's 12 transformer layers are unfrozen and updated end-to-end.
- The [CLS] token's final hidden state is fed through a linear classification head.
- Cross-entropy loss is minimised between predicted class probabilities and true labels.
- Gradients flow back through all layers, adapting BERT's language representations to the sarcasm detection task.

CHAPTER 7

EVALUATION

7.1 Methodology

After training, predictions are generated on the held-out test set using trainer.predict(). The raw logits are converted to predicted class labels by taking the argmax:

```
preds = trainer.predict(test_dataset)

y_pred = preds.predictions.argmax(axis=1)


print('Accuracy:', accuracy_score(test_labels, y_pred))

print('Classification Report:\n', classification_report(test_labels, y_pred))
```

7.2 Metrics Explained

Metric	Formula	Interpretation
Accuracy	Correct / Total	Overall fraction of correct predictions
Precision	TP / (TP + FP)	Of all predicted sarcastic, how many were actually sarcastic?
Recall	TP / (TP + FN)	Of all actual sarcastic, how many were correctly detected?
F1-Score	2 × (P × R) / (P + R)	Harmonic mean of precision and recall — best for imbalanced classes

CHAPTER 8

MODEL DETAILS — BERT ARCHITECTURE

8.1 Why BERT?

BERT (Bidirectional Encoder Representations from Transformers) was chosen because:

- It is pre-trained on massive corpora (Wikipedia + BookCorpus), giving it deep linguistic knowledge.
- Its bidirectional attention means it reads text left-to-right and right-to-left simultaneously, capturing full context.
- The bert-base-uncased variant is computationally efficient while maintaining strong NLP performance.
- HuggingFace's BertForSequenceClassification adds a ready-to-use classification head.

8.2 Architecture Summary

Component	Specification
Base Model	bert-base-uncased
Transformer Layers	12
Attention Heads	12
Hidden Size	768
Parameters	~110 million
Max Sequence Length	512 tokens

CHAPTER 9

LIMITATIONS & FUTURE WORK

9.1 Current Limitations

- Training for only 2 epochs may be insufficient for the model to fully converge on complex sarcasm patterns.
- No evaluation metric is computed during training (eval_strategy is not set), so training loss is the only indicator of progress.
- Fixed max_length=128 may truncate longer comments where sarcasm appears late in the text.
- No cross-validation; a single 80/20 split may not give a stable accuracy estimate.
- No data augmentation techniques are applied to handle potential class imbalance.

9.2 Recommended Improvements

Improvement	Description	Expected Benefit
Increase Epochs	Train for 3–5 epochs with early stopping	Higher accuracy, reduced underfitting
Add Evaluation Strategy	Set eval_strategy='epoch' to monitor val loss	Detects overfitting early
Experiment with Variants	Try RoBERTa, DeBERTa, or DistilBERT	Better accuracy or faster inference
Learning Rate Tuning	Grid search over 1e-5 to 5e-5	Improved convergence speed

CHAPTER 10

REPRODUCTION GUIDE

10.1 Environment Setup

```
# Install required packages

pip install torch transformers pandas scikit-learn



# Verify GPU availability (optional)

python -c "import torch; print(torch.cuda.is_available())"
```

10.2 Step-by-Step Execution

Step	Action	Expected Output
1	Load dataset from CSV	DataFrame with 'comment' and 'label' columns
2	Split data 80/20	train_texts, test_texts, train_labels, test_labels
3	Tokenise with BertTokenizer	input_ids, attention_mask tensors
4	Create SarcasmDataset objects	PyTorch Dataset instances
5	Load BertForSequenceClassification	Model with 2-class head
6	Configure TrainingArguments	Training hyperparameters object
7	Initialise and run Trainer	Training logs printed every 50 steps

Step	Action	Expected Output
8	Run predictions on test set	Logit array (N x 2)
9	Compute metrics	Accuracy + Classification Report

10.3 Expected Runtime

Hardware	Estimated Training Time (2 Epochs)
NVIDIA T4 / V100 GPU	10–25 minutes
Apple M1/M2 (MPS)	25–50 minutes
CPU only (modern)	2–6 hours